

Lecture 2

In Class Exercises



Exercise 1

- Consider the SUM-ARRAY procedure below:

```
SUM-ARRAY( $A, n$ )  
1   $sum = 0$   
2  for  $i = 1$  to  $n$   
3       $sum = sum + A[i]$   
4  return  $sum$ 
```

- State a loop invariant for this procedure, and use its **initialization**, **maintenance**, and **termination** properties to show that it returns the sum of the numbers in $A[1:n]$.

Exercise 2

- ❑ Consider sorting n numbers stored in array $A[1:n]$ by first finding the smallest element in $A[1:n]$ and exchanging it with the element in $A[1]$. Then find the smallest element of $A[2:n]$, and exchange it with $A[2]$, then find the smallest element of $A[3:n]$ and exchange it with $A[3]$. Continue in this manner for the first $n-1$ elements of A .
- ❑ Write the pseudocode for this algorithm.
- ❑ What loop invariant does this algorithm maintain?
- ❑ Why does it need to run for the first $n-1$ elements, rather than all n elements?
- ❑ Give the worst-case running time in Θ -notation.
- ❑ Is the best-case running time any better?

Exercise 3

- ❑ Input: A sequence of n numbers $\{a_1, a_1, \dots, a_n\}$ stored in array $A[1:n]$, a range [low:high] to search for and a value x .
- ❑ Output: An index i such that x equals $A[i]$ or the special value *NIL* if x does not appear in A .
- ❑ Assume that the array A is already sorted.
- ❑ Write the **binary search** algorithm which searches for x by halving the size of the remaining portion of the subarray each time. Write pseudocode, either iterative or recursive.
- ❑ Argue that the worst-case running time for binary search is $\Theta(\lg n)$.